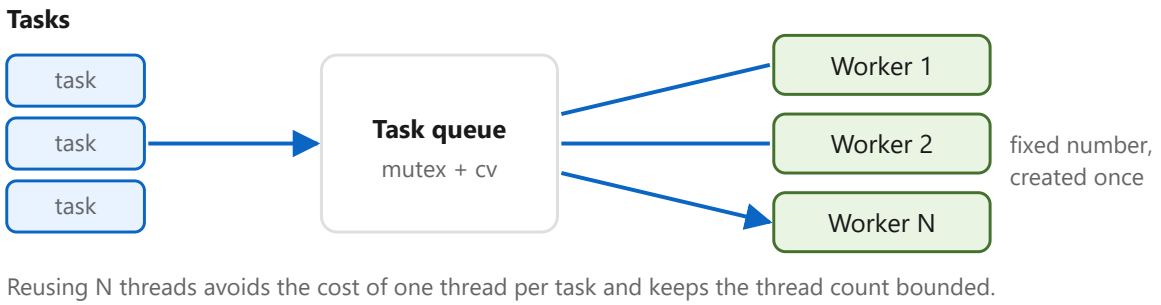
Thread Pools, Coordination and Parallel Algorithms

How to run many tasks on a few threads: the pool and its queue, sizing and the pool deadlock, work stealing, `latch`, `barrier`, `semaphore` and `stop_token`, and the C++17 parallel algorithms. Assumes Futures, async and Promises.

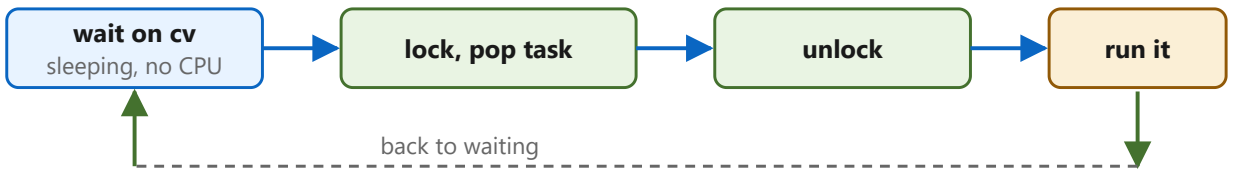
Thread pools and the task queue

Starting a thread per task is expensive, and thousands of threads overload the scheduler. A **thread pool** starts a fixed number of workers once and feeds them from a queue: the classic **producer/consumer** pattern.



Reusing N threads avoids the cost of one thread per task and keeps the thread count bounded.

Thread pool: many tasks, few long-lived workers.



stop flag set + queue empty -> the worker returns

If the task ran under the lock, only one task could run at a time.

submit(): push under the lock, then notify_one().

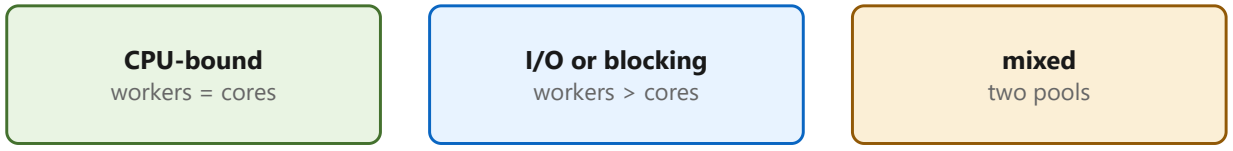
The loop of a pool worker: the task runs outside the lock.

```
// each worker runs this loop
workers_.emplace_back([this] {
    for (;;) {
        std::function<void()> task;
        {
            std::unique_lock lk(m_);
            cv_.wait(lk, [this] { return stop_ || !tasks_.empty(); });
            // finish the queue, then exit
            if (stop_ && tasks_.empty()) return;
            task = std::move(tasks_.front()); tasks_.pop();
        // unlock BEFORE running the task
        }
        task();
    }
});
```

The destructor sets `stop_` under the lock, calls `notify_all()` and joins every worker. It combines everything from the earlier lessons: mutex, condition variable, stop flag and RAII shutdown.

Sizing a pool and the pool deadlock

How many workers? It depends on what the tasks do while they run.



A task that waits for a subtask on the SAME pool can block every worker:

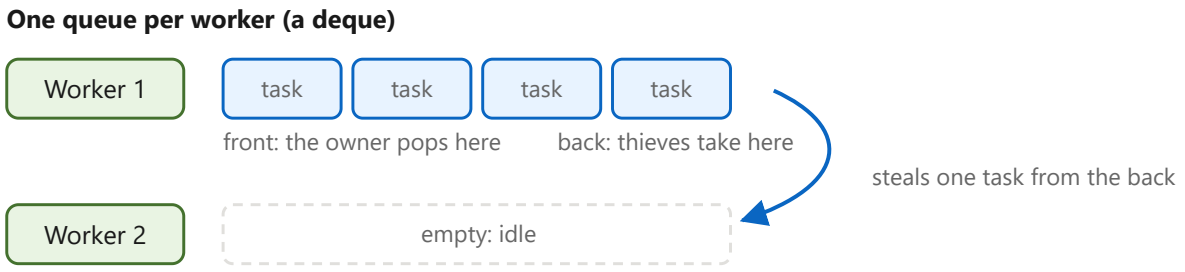
all N workers wait for subtasks that nobody is free to run.

Size a pool by what its tasks do.

WORK	WORKERS
CPU-bound	about `hardware_concurrency()`; more is oversubscription and context switches
I/O-bound or blocking	more than the core count, because threads spend their time waiting
Mixed	separate pools, so slow blocking tasks cannot starve the fast ones

> **POOL DEADLOCK** A task that submits a subtask to the same pool and **waits** for it can block every worker. Do not block a pool thread on work that needs the same pool.

Work stealing



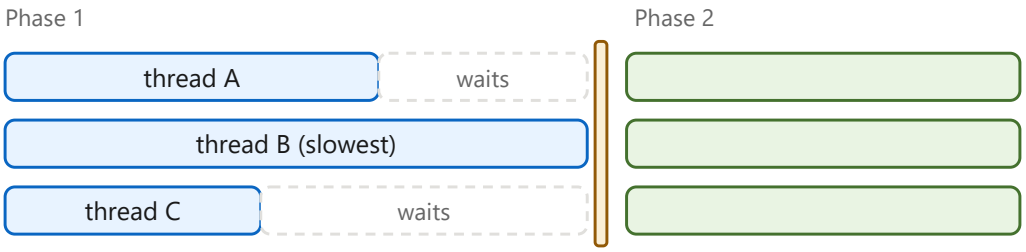
Owners and thieves work at opposite ends, so they rarely contend. A shared queue has one lock everyone fights over.

Work stealing: local deques keep the common case lock-light, stealing balances the load.

- **Work stealing** is how production pools (TBB, Java's fork/join) balance load: each worker pops its own deque and an idle worker **steals** from another's back.
- It suits **recursive** tasks that spawn subtasks: a spawned task goes on the spawner's own deque and stays hot in its cache.
- A simple pool with one shared queue (the previous section) is fine until that queue's lock becomes the bottleneck. Know that stealing is the next step; do not claim to have written one.

`latch`, `barrier` and `semaphore` (C++20)

TOOL	IDEA	REUSABLE?
`std::latch(n)`	wait until a counter has been counted down *n* times ("N things are done")	no: one shot
`std::barrier(n)`	all *n* threads wait for each other at the end of each phase, then all go on	**yes**: phase after phase
`std::counting_semaphore<N>`	at most *k* threads in a section (a permit counter)	yes
`std::binary_semaphore`	a semaphore with one permit: signal from one thread to another	yes



The barrier (vertical bar): the last arrival releases everyone, so phase 2 starts together.
Compiled: every thread saw all three phase-1 results before it began phase 2.

A barrier separates phases of a parallel computation.

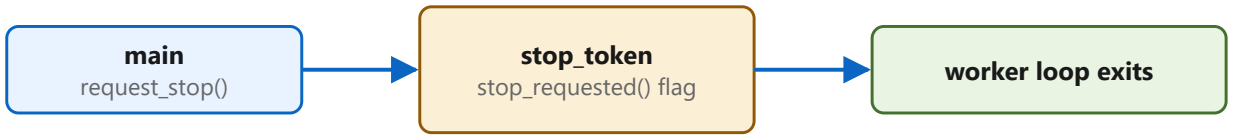
```
// three workers will count down
std::latch done(3);
for (int i = 0; i < 3; ++i) workers.emplace_back([&] { work();
done.count_down(); });
// returns when the count reaches 0
done.wait();

// three threads, many phases
std::barrier sync(3);
auto body = [&] { phase1(); sync.arrive_and_wait(); phase2(); };

// at most two threads at once
std::counting_semaphore<4> slots(2);
// compiled: peak was 2 with six threads
void limited() { slots.acquire(); use(); slots.release(); }
```

Cooperative cancellation: `jthread` and `stop_token`

You cannot safely kill a thread from outside, because it may hold a lock or be half way through an update. Instead you **ask** it to stop, and it checks regularly. `std::stop_token` is the standard channel for that request.



std::jthread: request_stop() then join() in its destructor: no forgot-to-join crash.

There is no safe way to kill a thread: you ask it to stop.

```
std::jthread worker([](std::stop_token st) {
    // check the token regularly
    while (!st.stop_requested()) doWork();
});
// worker.request_stop() or just let the destructor: request_stop() then
// join()
```

> **DESIGN FOR IT** Make long loops check the token (or a flag) once per iteration, and make blocking waits wake up for it: `condition_variable_any::wait(lk, token, pred)` does.

Parallel algorithms (C++17)

Most standard algorithms accept an **execution policy** as a first argument. The library decides how to split the work across threads.

```
#include <execution>

std::sort(std::execution::par, v.begin(), v.end());
// compiled: 500500 for 1..1000
long long sum = std::reduce(std::execution::par, v.begin(), v.end(), 0LL);
std::transform(std::execution::par_unseq, a.begin(), a.end(), out.begin(),
f);
```

POLICY	MEANING
`seq`	sequential, as without a policy
`par`	may run on several threads
`par_unseq`	several threads and vectorised (SIMD): the function must not lock or allocate

- **Your function object must be thread-safe**: no unsynchronised writes to shared data. A data race inside a parallel algorithm is still undefined behaviour.
- `std::reduce` (not `accumulate`) is the parallel fold: it may **reassociate** the operation, so the operation must be associative (floating-point sums may differ slightly).
- It only pays for **big inputs and heavy work**; for small ranges the thread overhead dominates.
- With g++ you usually need Intel TBB installed and `-ltbb` for real parallelism; without a parallel backend the same code runs serially (as in the compiled example here).
- Exceptions escaping the callback call `std::terminate`.

CHOOSE Parallel algorithms for data parallelism over a container; a pool or `std::async` for independent tasks; a queue and a pinned thread for a latency-critical pipeline.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Thread pools and the task queue	C++ Concurrency in Action	2.4 Choosing the number of threads at runtime	54 to 57
	C++ Concurrency in Action	9.1.1 The simplest possible thread pool	324 to 326
Sizing a pool and the pool deadlock	C++ Concurrency in Action	9.1.1 The simplest possible thread pool	324 to 326
Work stealing	C++ Concurrency in Action	9.1.1 The simplest possible thread pool	324 to 326